

Algoritmo Genetico no Diagnostico de Cancer de Mama em Mulheres

Tech Challenge Fase 2 - Projeto 1

Resumo

Este documento apresenta o desenvolvimento de uma solucao computacional de apoio ao diagnostico de cancer de mama em mulheres, com foco na otimizacao de hiperparametros de modelos de Machine Learning por meio de Algoritmos Geneticos. O projeto foi construido em Python, com arquitetura modular, testes automatizados, geracao de relatorios, visualizacoes graficas e uma camada desacoplada de LLM para explicacoes em linguagem natural.

O problema foi tratado como classificacao binaria, com prioridade para recall, uma vez que em cenarios clinicos de rastreio e apoio diagnostico o custo de falso negativo tende a ser mais alto do que o de falso positivo. Para isso, utilizou-se o dataset publico Breast Cancer Wisconsin Diagnostic, embarcado no `scikit-learn`. Na etapa de comparacao inicial, foram avaliados quatro modelos baseline. A etapa de otimizacao genetica passou a ser aplicada automaticamente sobre a familia de modelo mais performatica segundo a mesma funcao de fitness usada na busca evolutiva.

Tres experimentos com algoritmo genetico foram executados, variando populacao, taxa de mutacao, numero de geracoes e estrategia de selecao. A solucao final tambem incorporou interface web em Streamlit, API separada com FastAPI, geracao automatica de graficos, exportacao de artefatos, logging em arquivo e estrutura inicial para implantacao em nuvem.

1. Introducao

O uso de inteligencia artificial em saude exige equilibrio entre desempenho, interpretabilidade, responsabilidade etica e praticidade de uso. Em aplicacoes diagnosticas voltadas a saude da mulher, o sistema deve apoiar, e nao substituir, a avaliacao clinica. Dentro desse contexto, a otimizacao de hiperparametros por algoritmo genetico se mostra adequada porque permite explorar configuracoes de modelos de forma automatizada, mantendo foco em metricas clinicamente relevantes.

Neste projeto, o dominio escolhido foi o apoio ao diagnostico de cancer de mama. A proposta foi construir um pipeline capaz de:

- treinar e comparar multiplos modelos baseline;
- otimizar hiperparametros com algoritmo genetico;
- comparar resultados entre baseline e modelos otimizados;
- gerar explicacoes textuais para profissionais de saude;
- disponibilizar resultados em modo CLI, relatorio e interface web.

2. Objetivos

2.1 Objetivo geral

Desenvolver uma aplicacao em Python para apoio ao diagnostico de cancer de mama em mulheres, utilizando Machine Learning e Algoritmo Genetico para otimizacao de hiperparametros, com camada de explicabilidade textual via LLM.

2.2 Objetivos especificos

- utilizar um dataset publico e reproduzivel;
- construir um modelo base sem otimizacao;
- implementar algoritmo genetico completo para busca de hiperparametros;
- executar ao menos tres experimentos distintos;
- priorizar recall na funcao fitness;
- disponibilizar explicacoes clinicas em linguagem natural;
- registrar saidas em JSON/JSONL;
- manter arquitetura organizada, testavel e documentada.

3. Dataset Utilizado

O dataset empregado foi o Breast Cancer Wisconsin Diagnostic, disponibilizado no pacote `scikit-learn`.

Caracteristicas relevantes

- total de amostras: 569;
- problema de classificacao binaria;
- contexto: apoio ao diagnostico de lesoes mamarias;
- alvo ajustado no projeto: 1 = maligno, 0 = benigno.

No projeto, o alvo original foi invertido para que a classe positiva representasse malignidade, o que torna o recall diretamente alinhado ao objetivo clinico de reduzir falsos negativos.

4. Arquitetura da Solucao

A aplicacao foi organizada em modulos coesos, com separacao clara entre configuracao, dados, avaliacao, algoritmo genetico, LLM, visualizacao, experimentacao, API e camada de apresentacao.

Estrutura principal

src/diagnostico_saude_mulher/
config/
data/
evaluation/
experiments/
api/
genetic_algorithm/
llm/
models/
ui/
utils/
visualization/

Pontos de entrada

- main.py: execucao principal do pipeline;
- run_genetic_optimization.py: experimentacao genetica com geracao de graficos;
- app.py: interface web com Streamlit;
- api.py: API HTTP para integracao futura e preparacao para a Fase 3.

5. Metodologia de Machine Learning

Na etapa de baseline, foram avaliados quatro algoritmos: RandomForestClassifier, LogisticRegression, DecisionTreeClassifier e KNeighborsClassifier. O modelo de referencia para a etapa genetica nao e fixado manualmente. Ele e selecionado automaticamente pelo melhor valor de fitness em validacao cruzada, o que torna a etapa evolutiva coerente com o baseline mais promissor.

Avaliacao

O pipeline calcula:

- recall;
- especificidade;
- F1-score;
- acuracia;
- precisao;
- ROC AUC;
- gap de equidade, quando houver atributo apropriado.

Foi utilizada validacao cruzada estratificada para reduzir o risco de superajuste durante a avaliacao da funcao de fitness.

6. Algoritmo Genetico

O modulo genetico foi implementado de forma completa, com os seguintes elementos:

- representacao genetica dos hiperparametros;
- populacao inicial aleatoria;
- funcao fitness ponderada;
- selecao por torneio e por roleta;
- crossover uniforme gene a gene;
- mutacao aleatoria por gene;
- elitismo;
- historico de melhores individuos por geracao.

Espaco de busca

O espaco de busca depende da familia de modelo escolhida pelo fitness. Na execucao atual, como a LogisticRegression foi selecionada como referencia, os genes considerados pelo AG foram:

- classifier__C
- classifier__solver
- scaler__with_mean
- scaler__with_std

Funcao fitness

A funcao de fitness prioriza recall, sem ignorar a necessidade de controle de falsos positivos e estabilidade preditiva:

- 55% recall
- 25% especificidade
- 20% F1-score
- penalizacao opcional por gap de equidade

Essa composicao foi escolhida por refletir o risco clinico maior associado a casos malignos nao identificados.

7. Integracao com LLM

O sistema possui uma camada desacoplada de LLM, permitindo trocar o provedor sem alterar o restante da aplicacao.

Modos suportados

- mock: resposta deterministica para testes e uso local;
- http: integracao com endpoint compativel com chat completions.

Caracteristicas da explicacao gerada

- classificacao prevista;
- probabilidade estimada;
- interpretacao cautelosa;
- orientacoes para profissional de saude;
- linguagem sensivel a genero;
- aviso explicito de que o sistema nao substitui avaliacao medica.

As respostas sao registradas em `artifacts/llm_responses.jsonl`, o que tambem permite auditoria, historico na interface e reuso futuro.

8. Experimentos Realizados

Foram executados tres experimentos geneticos:

1. Populacao 6, geracoes 3, mutacao 0.10, selecao tournament
2. Populacao 8, geracoes 4, mutacao 0.15, selecao tournament
3. Populacao 10, geracoes 5, mutacao 0.20, selecao roulette

9. Resultados Obtidos

Os resultados abaixo foram extraidos da execucao real registrada em `artifacts/experimentos_geneticos.json`.

9.1 Comparacao entre modelos baseline

Antes da etapa genetica, o projeto comparou quatro modelos supervisionados no mesmo split de treino e teste.

Modelo	Recall	Especificidade	F1-score	Acuracia
RandomForestClassifier	92,86%	98,61%	95,12%	96,49%
LogisticRegression	95,24%	98,61%	96,39%	97,37%
DecisionTreeClassifier	85,71%	94,44%	87,80%	91,23%

Modelo	Recall	Especificidade	F1-score	Acuracia
KNeighborsClassifier	90,48%	98,61%	93,83%	95,61%

Leituras principais:

- LogisticRegression apresentou o melhor recall e o melhor F1 entre os baselines avaliados;
- LogisticRegression tambem obteve o maior valor de fitness em validacao cruzada e, por isso, foi selecionada como referencia para o AG;
- DecisionTreeClassifier apresentou o pior equilibrio geral;
- KNeighborsClassifier teve boa especificidade, mas ficou abaixo de LogisticRegression e RandomForest em recall.

9.2 Baseline de referencia para o AG

Validacao cruzada

- recall: 96,47%
- especificidade: 97,89%
- F1-score: 96,47%
- ROC AUC: 99,49%

Teste

- recall: 95,24%
- especificidade: 98,61%
- F1-score: 96,39%
- acuracia: 97,37%
- precisao: 97,56%
- ROC AUC: 99,54%

Esse baseline de referencia corresponde a LogisticRegression, selecionada automaticamente como ponto de partida para a etapa de otimizacao genetica.

9.3 Experimento genetico 1

Melhores hiperparametros

- classifier__C=0.1
- classifier__solver=lbgfs
- scaler__with_mean=True
- scaler__with_std=True

Teste

- recall: 97,62%
- especificidade: 100,00%
- F1-score: 98,80%
- acuracia: 99,12%

9.4 Experimento genetico 2

Melhores hiperparametros

- classifier__C=0.1
- classifier__solver=lbfgs
- scaler__with_mean=True
- scaler__with_std=True

Teste

- recall: 97,62%
- especificidade: 100,00%
- F1-score: 98,80%
- acuracia: 99,12%

9.5 Experimento genetico 3

Apresentou o mesmo conjunto de hiperparametros vencedores dos experimentos anteriores, com diferenca apenas no numero de geracoes executadas.

9.6 Analise comparativa

Os baselines ja apresentaram desempenho forte, com destaque para LogisticRegression. Como essa familia foi a mais performatica pela funcao de fitness, ela tambem se tornou a base do AG. Nesse contexto, o algoritmo genetico elevou o desempenho do modelo de referencia e encontrou combinacoes que:

- aumentaram o recall de teste;
- elevaram a especificidade para 100,00%;
- elevaram F1-score e acuracia para o melhor nivel observado no projeto.

Isso indica que, no contexto deste dataset, o AG foi eficaz em melhorar o equilibrio da propria LogisticRegression, porque a familia vencedora no baseline tambem foi a familia otimizada.

10. Hiperparametros Otimizados Encontrados

Os melhores conjuntos de hiperparametros encontrados pelo algoritmo genetico foram os seguintes:

Experimento 1

- `classifier__C = 0.1`
- `classifier__solver = lbfgs`
- `scaler__with_mean = True`
- `scaler__with_std = True`

Experimento 2

- `classifier__C = 0.1`
- `classifier__solver = lbfgs`
- `scaler__with_mean = True`
- `scaler__with_std = True`

Experimento 3

- `classifier__C = 0.1`
- `classifier__solver = lbfgs`
- `scaler__with_mean = True`
- `scaler__with_std = True`

Em termos praticos, qualquer um dos tres experimentos apresentou o mesmo conjunto de hiperparametros vencedores para a LogisticRegression, com recall de 97,62%, especificidade de 100,00%, F1-score de 98,80% e acuracia de 99,12% no conjunto de teste.

10.1 Impacto dos hiperparametros no fitness

Para tornar a analise mais objetiva, a tabela abaixo resume como cada melhor configuracao se comportou na fase de busca do algoritmo genetico.

Experimento	C	Solver	with_mean	with_std	Fitness final	Recall CV	Especificidad e CV	F1 CV
AG 1	0,1	lbfgs	True	True	0,9712	0,9647	0,9860	0,9704
AG 2	0,1	lbfgs	True	True	0,9712	0,9647	0,9860	0,9704
AG 3	0,1	lbfgs	True	True	0,9712	0,9647	0,9860	0,9704

Leitura tecnica dos resultados:

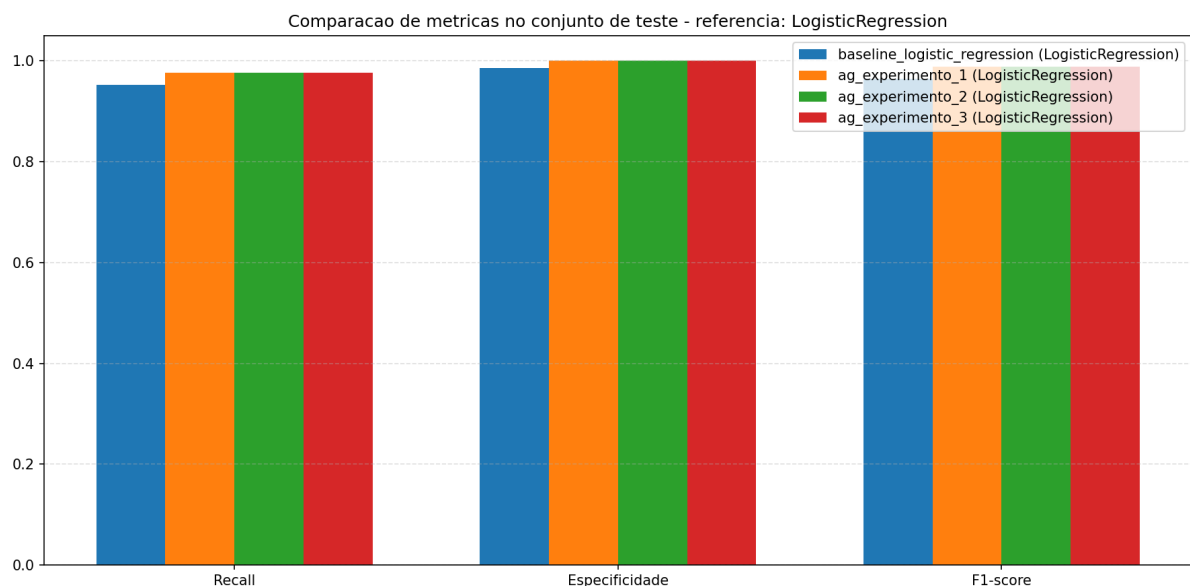
- os tres experimentos convergiram para a mesma configuracao vencedora da LogisticRegression;
- o melhor valor de fitness encontrado foi 0,9712, acima do baseline de referencia em validacao cruzada;
- isso sugere que o espaco de busca da regressao logistica, neste dataset, tem uma regio otima bastante dominante;
- por esse motivo, os graficos atuais mostram convergencia rapida e baixa dispersao entre experimentos.

11. Visualizacoes e Graficos

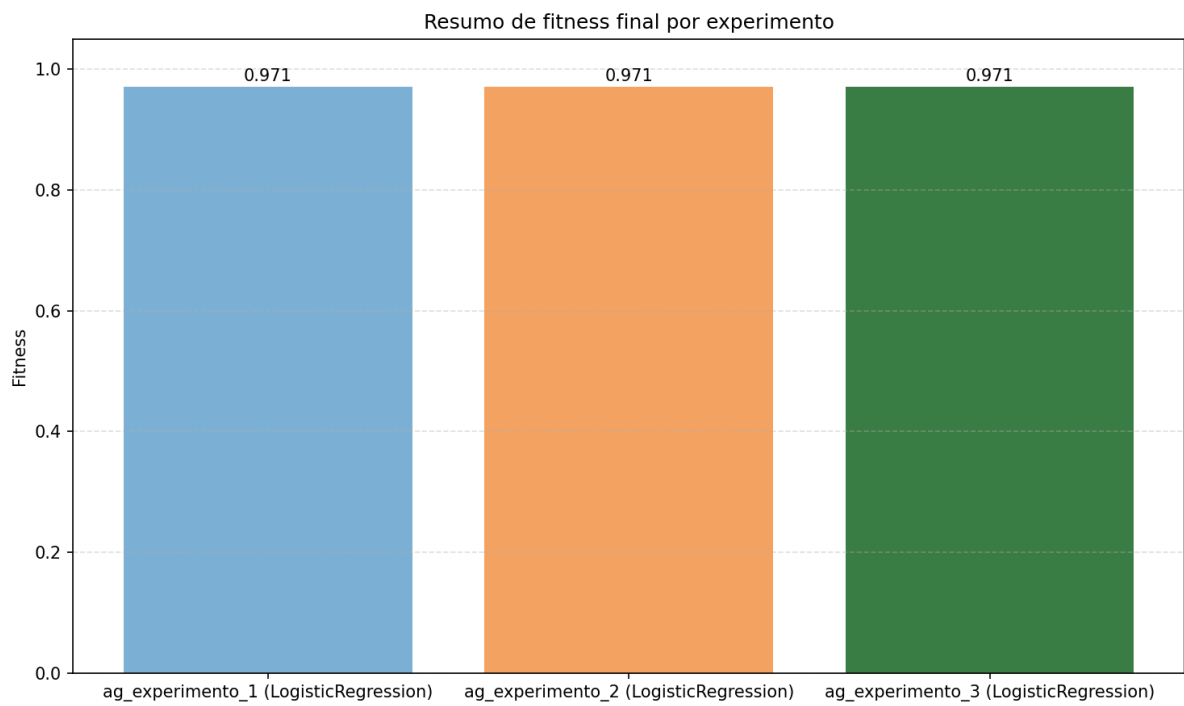
Os graficos abaixo foram gerados automaticamente pelo projeto e estao incorporados ao relatorio.

O projeto gera automaticamente:

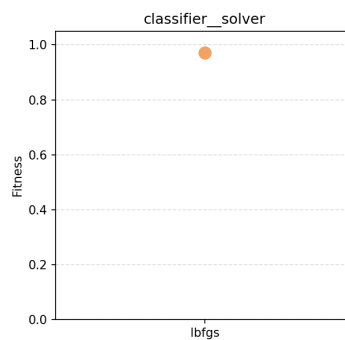
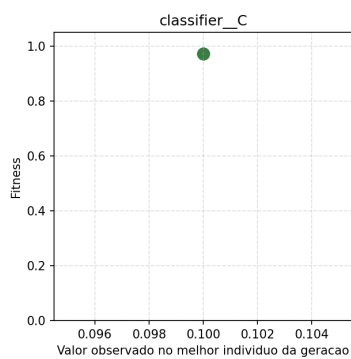
- Comparacao de metricas



- Resumo dos experimentos

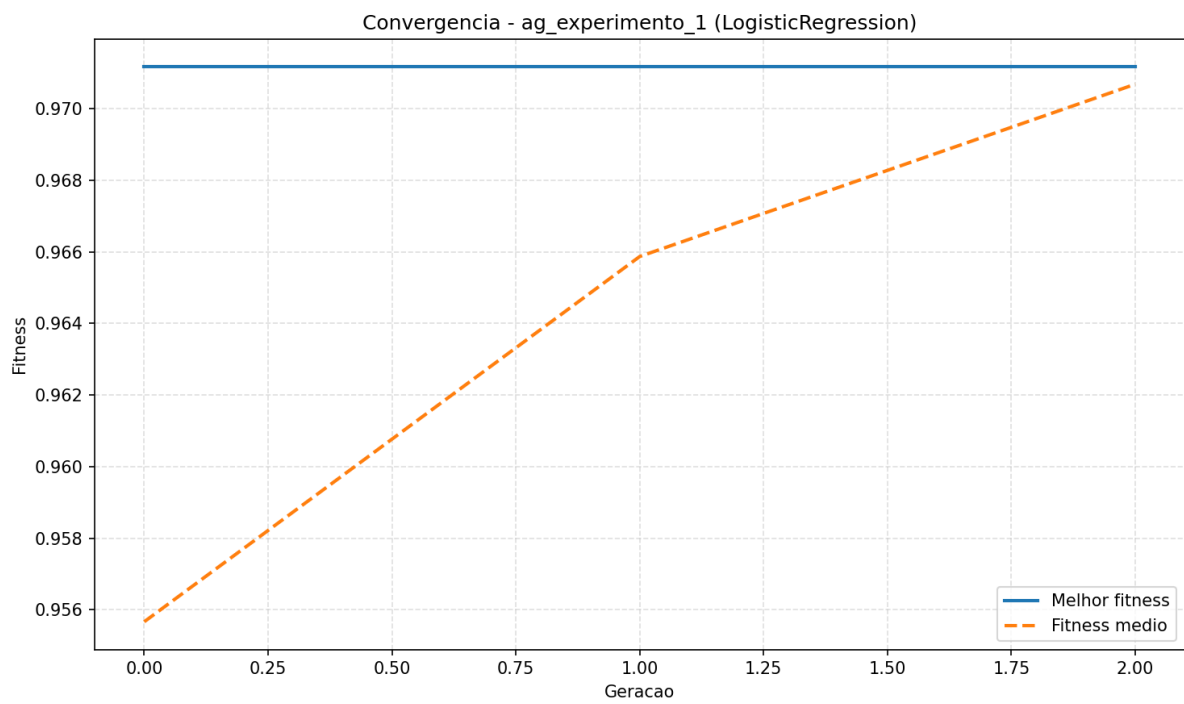


- Comportamento do fitness com os hiperparametros

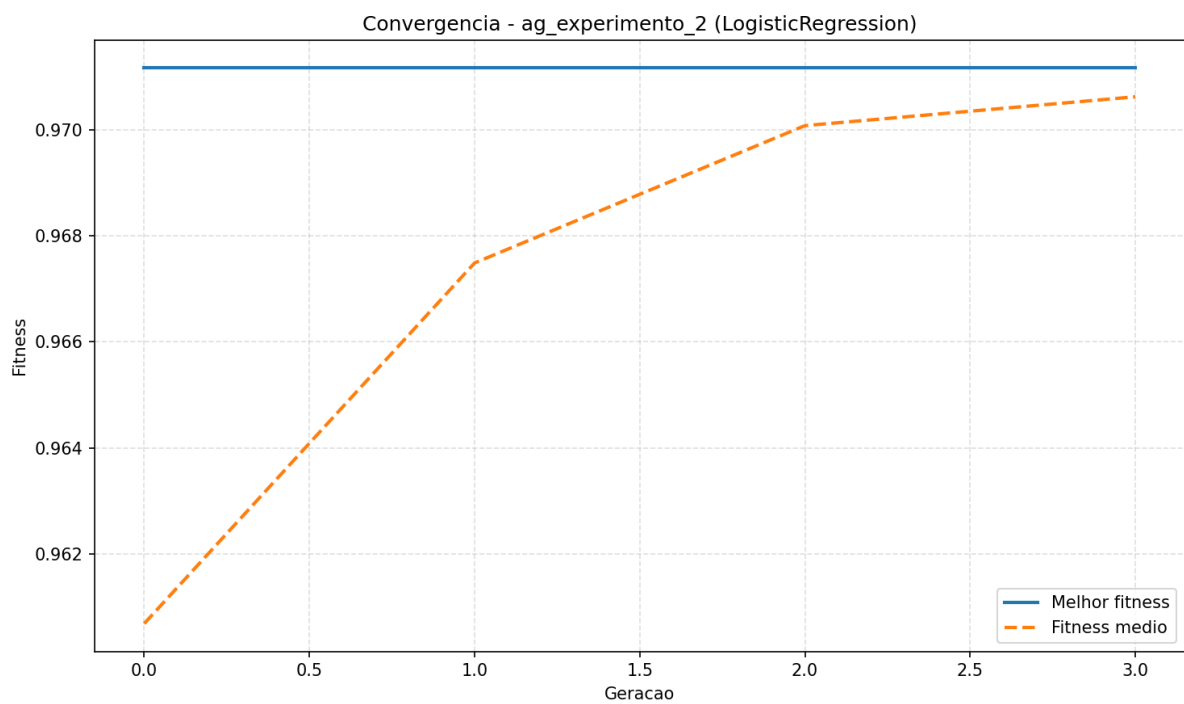


Leitura:
 - Cada ponto representa o melhor individuo de uma geracao.
 - O eixo Y mostra o fitness observado naquela geracao.
 - O modelo otimizado nesta execucao foi: LogisticRegression.

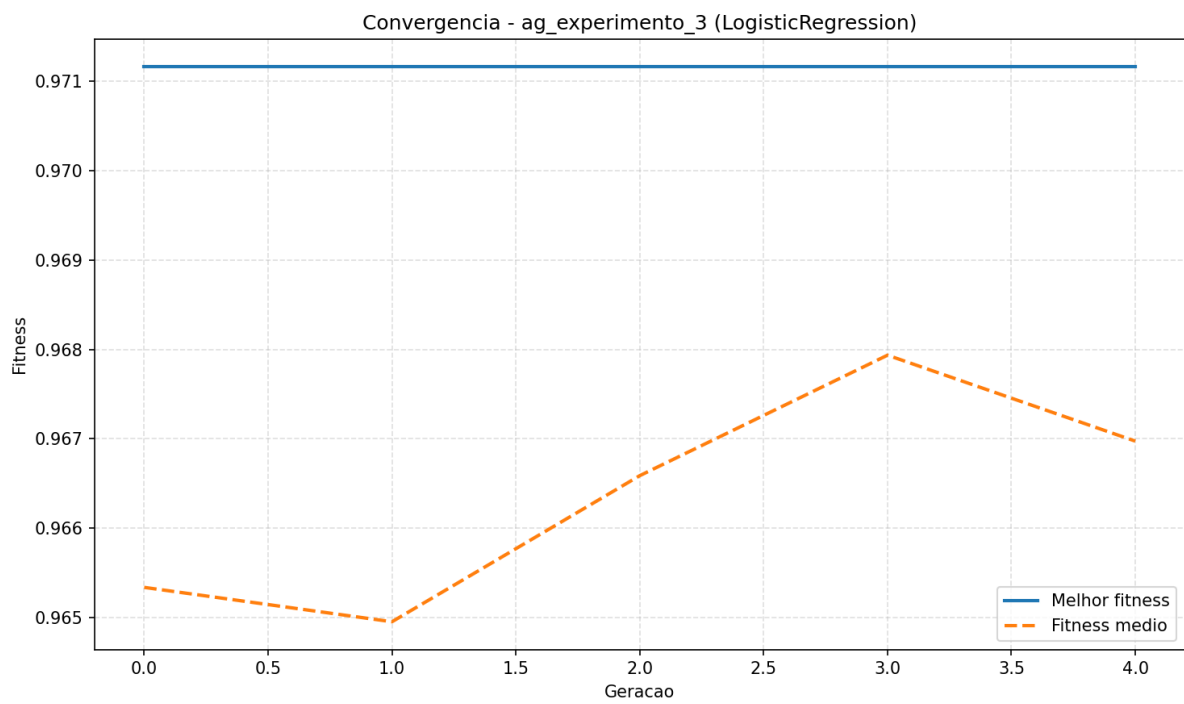
- Convergencia do experimento 1



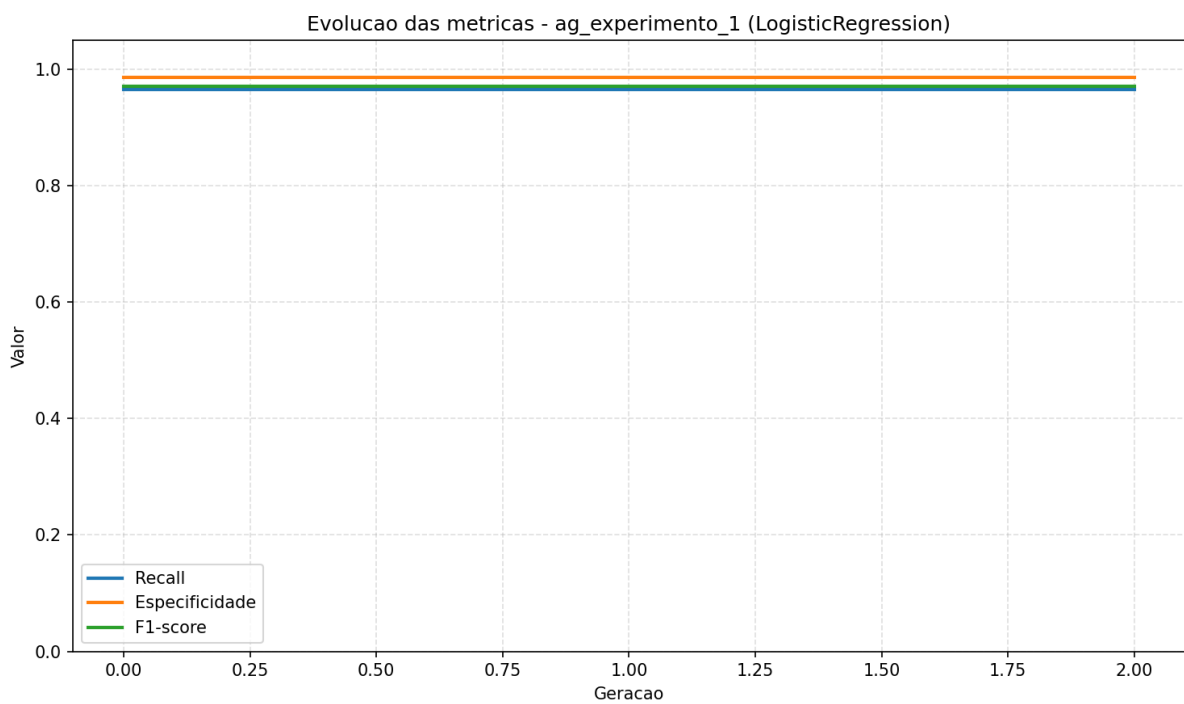
- Convergencia do experimento 2



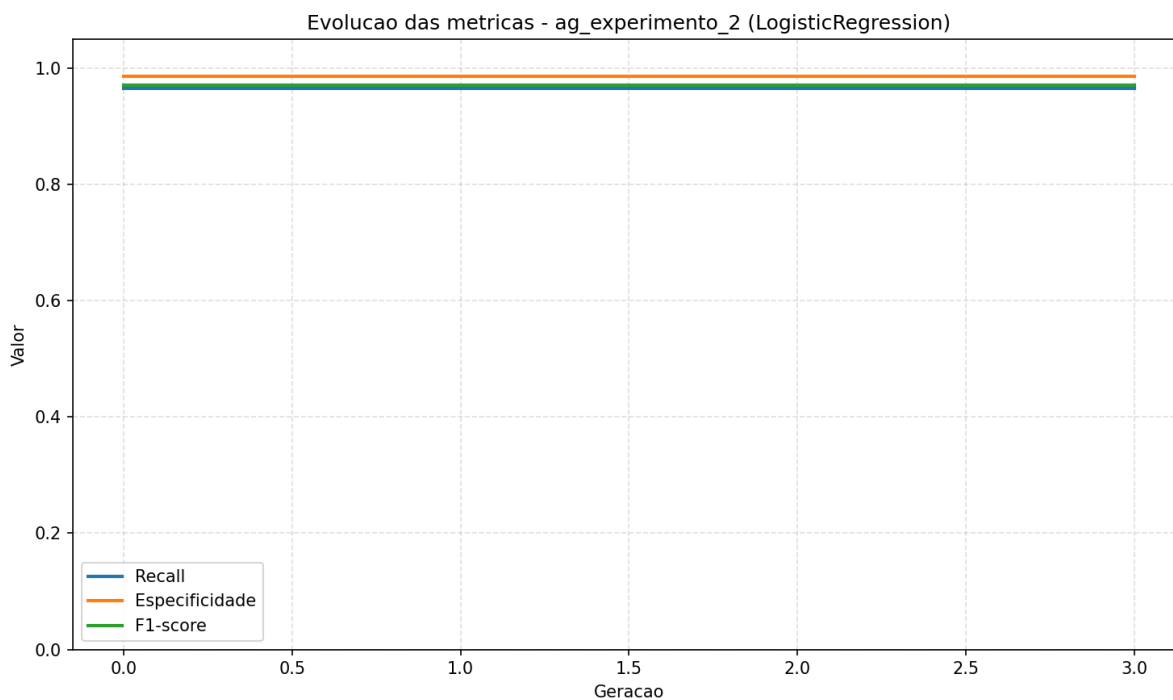
- Convergencia do experimento 3



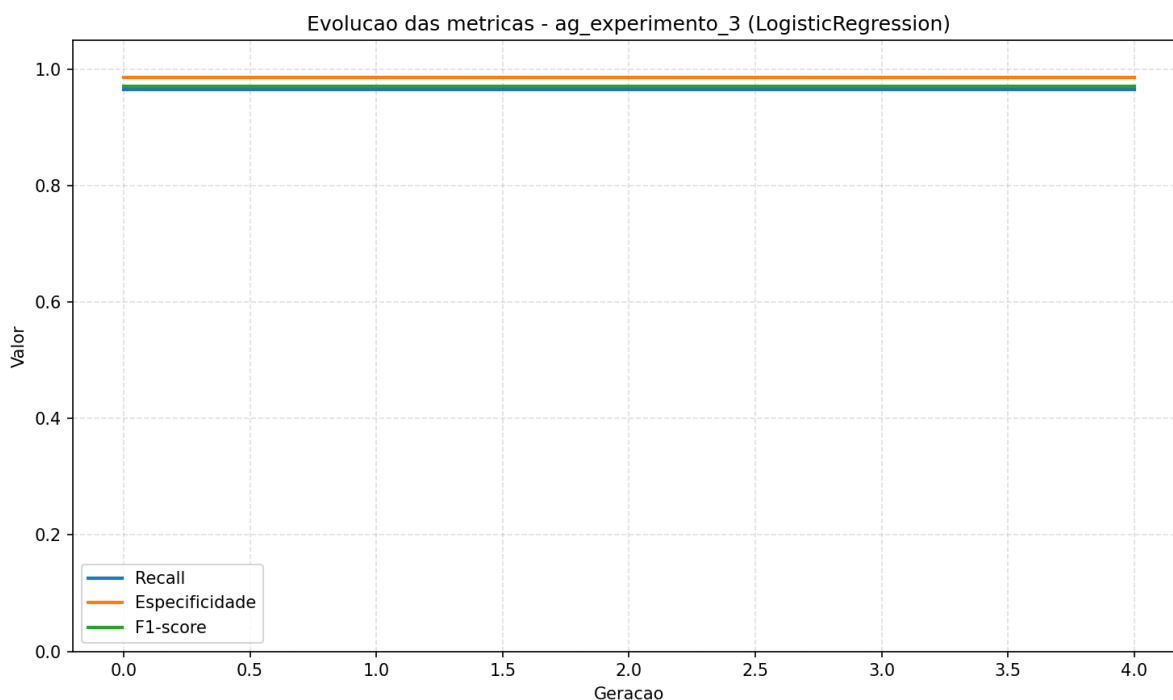
- Metricas por geracao - experimento 1



- Metricas por geracao - experimento 2



- Metricas por geracao - experimento 3



Esses graficos apoiam a analise da estabilidade do AG e a comparacao entre baseline e modelos otimizados.

Analises adicionais

Para aprofundar a interpretacao dos resultados, o projeto inclui analises adicionais sobre o comportamento do modelo:

- variacao do fitness em funcao dos hiperparametros otimizados;

- evolucao de recall, especificidade e F1-score ao longo das geracoes;
- comparacao consolidada entre os melhores individuos de cada experimento.

Essas visualizacoes ajudam a responder nao apenas qual configuracao venceu, mas tambem como o modelo se comportou quando os hiperparametros variaram entre os experimentos.

12. Interface Web

Foi desenvolvida uma interface web em Streamlit para tornar o projeto mais demonstravel e mais proximo de um uso pratico.

Funcionalidades

- visualizacao do baseline e dos experimentos geneticos;
- comparacao de metricas;
- exibicao de graficos;
- carregamento do dataset padrao ou envio de CSV pelo usuario;
- simulacao de predicao em amostras do conjunto de teste;
- geracao de explicacao textual com a LLM configurada;
- exibicao do historico das respostas geradas pela LLM;
- aba de monitoramento com leitura dos logs da aplicacao.

13. API e Preparacao para a Fase 3

O projeto passou a expor uma API separada baseada em FastAPI, com o objetivo de preparar a arquitetura para cenarios futuros de integracao assincrona, servicos desacoplados e execucao em background.

Endpoints principais

- GET /health
- GET /dataset
- POST /baseline
- POST /optimize
- POST /predict
- GET /llm/history
- GET /logs

Essa separacao entre interface, API e dominio torna o projeto mais aderente a uma evolucao natural para a Fase 3.

14. Monitoramento e Logging

Foi implementado logging em arquivo com persistencia em:

- `artifacts/logs/aplicacao.log`

Os logs registram:

- inicio e fim do treinamento dos modelos;
- metricas relevantes do conjunto de teste;
- execucao do algoritmo genetico;
- desempenho por geracao;
- geracao e persistencia das respostas da LLM.

Esse material pode ser usado tanto para depuracao quanto para monitoramento operacional futuro.

15. Infraestrutura, Escalabilidade e Nuvem

O projeto foi preparado para execucao via container, com:

- `Dockerfile`
- `docker-compose.yml`

Tambem foi adicionada uma estrutura inicial de Infraestrutura como Codigo em Terraform:

- `infra/terraform/main.tf`
- `infra/terraform/variables.tf`
- `infra/terraform/outputs.tf`

Estrategias de escala

- separar a interface da execucao da otimizacao genetica;
- mover experimentos pesados para workers ou jobs assíncronos;
- manter artefatos fora do container, em armazenamento externo;
- expor a API como camada de integracao entre frontend e processamento.

16. Testes Automatizados

O projeto possui testes automatizados para:

- funcao fitness;
- operadores geneticos;

- treinamento basico do modelo;
- integracao do pipeline principal;
- exportacao de artefatos;
- geracao de graficos;
- prompts e cliente mock da LLM;
- helpers de UI;
- estrutura da API.

Na validacao mais recente, a suite executou 17 testes entre nucleo, visualizacao, integracao, API e helpers de apresentacao.

17. Consideracoes Eticas

Em saude, desempenho numerico nao e suficiente. Por isso, o projeto incorpora cuidados eticos explicitos:

- o sistema e de apoio, nao de decisao autonoma;
- a explicacao textual reforca limites do modelo;
- nao ha armazenamento de chaves de API no codigo;
- o pipeline foi preparado para equidade, embora o dataset atual nao permita auditoria demografica robusta;
- priorizar recall e uma escolha tecnica coerente com o risco de subdiagnostico.

18. Limitacoes

- o dataset e academico e nao substitui validacao clinica real;
- o projeto utiliza um conjunto de dados relativamente pequeno;
- a melhoria do AG no conjunto de teste foi mais evidente em especificidade e F1 do que em recall;
- a analise de equidade fica limitada pela ausencia de atributos demograficos adequados no dataset escolhido;
- a API ainda esta em modo inicial, sem autenticacao e sem fila de execucao.

19. Conclusao

O projeto atingiu os objetivos propostos ao construir uma solucao completa para apoio ao diagnostico de cancer de mama em mulheres, utilizando Machine Learning, Algoritmo Genetico e LLM.

Do ponto de vista tecnico, a solucao entregou:

- arquitetura limpa e modular;

- baseline forte e reproduzível;
- algoritmo genético funcional e auditável;
- comparação entre experimentos;
- geração automática de relatórios e gráficos;
- interface web de demonstração;
- camada de explicação textual desacoplada;
- API separada para integração futura;
- logs, Docker e IaC inicial.

Mesmo quando o ganho em recall no conjunto de teste não superou o baseline, o processo de experimentação mostrou valor ao identificar configurações com melhor equilíbrio geral entre recall, especificidade e F1-score. A arquitetura final também deixa a base preparada para a Fase 3, tanto do ponto de vista técnico quanto operacional.

20. Referencias

- scikit-learn: Breast Cancer Wisconsin Diagnostic Dataset
- documentação oficial do scikit-learn
- artefatos do próprio projeto em `artifacts/`
- código-fonte e documentação em `README.md` e `docs/architecture.md`
- arquitetura de nuvem em `docs/cloud_architecture.md`

21. Como converter este documento para PDF

Caso você queira um arquivo PDF final com esse conteúdo, uma opção simples no seu ambiente é:

1. abrir este arquivo em um editor com suporte a Markdown;
2. exportar para PDF;
3. ou usar uma ferramenta como Pandoc no seu ambiente local.